An Axisymmetric Vacuum Kerr BSSN Code

A complete description of the mathematics, the discretisation, the boundaries, the diagnostics, and the failure modes

GR/BSSN

Preface

This document explains, in detail, a three-dimensional-code-written-as-two-dimensional numerical relativity code that evolves the vacuum Kerr spacetime in the BSSN formulation, restricted to axisymmetry about the z-axis and discretised on a Cartoon grid. The intent is not merely to describe the code but to teach the underlying subject: after reading it you should understand not just what each routine does, but *why* it exists, what the alternatives were, what the traps are, and enough of the design that you could write your own evolution code from a blank file.

Every equation in this document was transcribed from the source, not from memory or from the literature. Where the code deviates from textbook form the deviation is called out explicitly, because in every such case the deviation was deliberate and has a reason. Where the code is known to be wrong, that is stated too; several such cases are documented, and two of them were found only by the investigation recorded in Chapter 9.

A warning about the current state. This code does *not* currently produce stable long-duration runs. It evolves exact Kerr–Schild data and develops an $O(1)$ Hamiltonian-constraint violation within one or two cells of the inner excision boundary, which grows until the conformal metric loses positive definiteness and the run aborts, typically at $t \approx 5.6$ in the production configuration. Chapter 9 documents the full investigation: what was ruled out, what was found, and what remains open. A large fraction of that chapter is about measurement errors that produced confident but wrong conclusions, which is itself instructive.

Contents

Preface **2**

1 Orientation **1**
1.1 What this code is . . . 1
1.2 Why the design is shaped this way . . . 1
1.3 Quick start . . . 1
1.4 Repository map . . . 2
1.4.1 Core evolution path . . . 2
1.4.2 Not on the production path . . . 3
1.4.3 Tests and diagnostics . . . 3

2 The Physics **5**
2.1 From ADM to BSSN . . . 5
2.1.1 ADM . . . 5
2.1.2 The conformal split . . . 5
2.2 Constraints . . . 6
2.3 Gauge . . . 6

3 The Discretisation **8**
3.1 The grid . . . 8
3.2 The Cartoon method in full . . . 8
3.2.1 What is stored . . . 8
3.2.2 The radial interpolation . . . 8
3.2.3 Rotation . . . 9
3.3 Finite differences and the fallback cascade . . . 9
3.3.1 Every stencil in the code . . . 9
3.3.2 The fallback logic . . . 10
3.3.3 The variable mask contract . . . 10

4 Boundaries **11**
4.1 Inner: excision . . . 11
4.1.1 The mask . . . 11
4.1.2 The ghost fill . . . 11
4.1.3 Onion-peeling and the frozen snapshot . . . 12
4.1.4 Positivity enforcement . . . 12
4.1.5 The characteristic SAT . . . 12
4.2 Outer: Sommerfeld . . . 13
4.3 Time integration . . . 13

4.3.1 The timestep . . . 13
4.3.2 The algebraic projection . . . 13

5 Initial Data **15**
5.0.1 The conformal connection . . . 15

6 Constraints and Diagnostics **16**
6.1 The margin trap . . . 16
6.2 The near-mask band . . . 16
6.3 The diagnostic programs . . . 17
6.3.1 diagnose-ghost-accuracy . . . 17
6.3.2 diagnose-momentum-point . . . 18
6.3.3 diagnose-inner-boundary . . . 18
6.3.4 A methodological warning . . . 18

7 Build and Test **19**
7.1 The parity gate . . . 19
7.2 Sources must be listed in dependency order . . . 19
7.3 Environment overrides . . . 20

8 A Tour of Every File **21**
8.0.1 The first-order track . . . 22
8.1 SBP-SAT: why it is not finished . . . 22

9 Findings **23**
9.1 The failure . . . 23
9.2 What was established . . . 23
9.2.1 Negative results, all measured . . . 23
9.2.2 The live picture . . . 24
9.3 Excision order: a trade-off, not a win . . . 24
9.4 How this investigation went wrong, repeatedly . . . 24

10 Writing Your Own **26**
10.1 Where to start . . . 26
10.2 Design lessons worth stealing . . . 27
10.3 Where the physics is genuinely unfinished . . . 27

11 Deriving BSSN, and Why It Exists **28**
11.1 The failure of ADM in practice . . . 28
11.2 What the constraint propagation mechanism needs . . . 28
11.3 Where each BSSN piece comes from . . . 29
11.4 Why the code writes $\tilde{R}$ in connection form . . . 29

12 A Worked Example: the Cartoon Method **31**
12.1 The problem . . . 31
12.2 Step 1: establish the symmetry correctly . . . 31
12.3 Step 2: reduce to radius and interpolate . . . 31
12.4 Step 3: rotate by rank . . . 32
12.5 Step 4: validate, in this order . . . 32

13 Debugging Numerical Relativity Code **33**
13.1 Rule 1: verify the instrument before interpreting it . . . 33
13.2 Rule 2: cross-check against an independent truth . . . 33
13.3 Rule 3: check the order of accuracy you claim . . . 33
13.4 Rule 4: know which knobs are inert . . . 34
13.5 Rule 5: gates must be able to fail . . . 34
13.6 Rule 6: keep a dated findings log, including retractions . . . 34
13.7 Rule 7: restate the baseline whenever anything moves . . . 34

14 Where the Numbers Come From **35**
14.1 The baseline of record . . . 35
14.2 Lifetimes . . . 35
14.3 Near-mask measurements . . . 35
14.4 Driver worst-point locations, corrected filter . . . 36

A Traps, Collected **37**

B Environment Overrides and Reproduction **38**

Chapter 1

Orientation

1.1 What this code is

The code evolves the vacuum Kerr spacetime in the **BSSN** formulation of general relativity, under the assumption of **axisymmetry about the z-axis**, using the **Cartoon method** to represent the y-direction derivatives, and **excising** a region inside the inner horizon so that no boundary condition is ever needed at the black hole.

The test problem is exact: Kerr–Schild initial data, $M = 1$, $a = 0.8$. Because this is an exact vacuum solution, the continuum constraints vanish identically everywhere. Any constraint violation the code reports is therefore, by construction, *pure discretisation error*. This makes the code an unusually clean testbed: there is no physics to blame.

The domain is the half-plane $x \in [0, x_{\max}]$, $z \in [-z_{\max}, z_{\max}]$, with the y-direction reconstructed on demand.

1.2 Why the design is shaped this way

Three decisions dominate the architecture, and each exists for a good reason:

1. **BSSN rather than ADM.** ADM's hyperbolic, constraint-preserving properties degrade badly in practice; BSSN adds 21 extra variables (a conformal factor, a traceless metric, a connection, a driver) and buys a genuinely constraint-satisfying evolution. The cost is stiffness and a much larger set of equations to discretise consistently.

2. **A Cartoon grid rather than full 3D.** A full N^3 grid is wasteful when the solution is axisymmetric. The Cartoon method stores only $y = 0$, $x \geq 0$ and synthesises the rest by interpolating in radius and rotating. The price is that y-derivatives are only as good as the radial interpolation, which in practice limits them to second order.

3. **Excision rather than a puncture condition.** Rather than imposing regularity at a coordinate singularity, the code simply never evolves points inside a surface $r_{\mathrm{KS}} < r_{\mathrm{excise}}$ and fills them by extrapolation. The price is that stencils near the mask degrade, and that degradation is where this code's problems live.

1.3 Quick start

```
# Build the production binary and run the full test suite (the gate).
make check

# Run the production configuration.
make run

# Run with runtime overrides, keeping output out of the tree.
make run RUN_ENV='GR_NSTEPS=700 GR_HORIZON_EVERY=0 GR_HZ_ABORT=0' RUN_DIR=/tmp

# Run a different parameter file.
make run PARFILE=parameter_files/vacuum_kerr_192.par
```

A typical short run looks like:

```
$ GR_NSTEPS=700 GR_HORIZON_EVERY=0 ./bin/vacuum_kerr parameter_files/vacuum_kerr.par
 Grid: Nx=48  Nz=96
 dx= 0.255  dy= 0.255  dz= 0.253
 Excision radius r_excise= 1.000  (horizon at r_+= 1.600M; interior frozen)
 step   700  t=  0.6252  max|H|(ext)=  1.2656E-02  rms|H|(ext)=  2.0211E-03
   max|M|(ext)=  2.8728E-02  alpha(probe)=  0.910348
    near-mask r_KS <   3.0000E+00  n=   192  max|H|=  1.9493E+00
      max|M|=  1.4920E+00  worst r=  1.0227E+00
```

Notice the near-mask line. The `max|H|(ext)` and `near-mask max|H|` differ by a factor of 154, and the two numbers are about completely different things. Chapter 6 explains why both exist, and why the first one is structurally incapable of seeing the region that matters.

1.4 Repository map

1.4.1 Core evolution path

These files, and only these, constitute the production solver:

File	Role
`helper/kinds\_mod.F90`	The single kind parameter. Eight lines.
`init/param.F90`	All runtime parameters and their environment overrides.
`init/grid.F90`	Grid coordinate arrays.
`init/vars.F90`	The state vector layout: 24 indices and their meanings.
`init/init\_data.F90`	Kerr–Schild initial data and the conformal connection.
`helper/cartoon.F90`	The Cartoon method: interpolation and rotation.
`helper/stencil.F90`	All finite differences, with excision-aware fallbacks.
`helper/geometry.F90`	Conformal geometry: Ricci tensors, Laplacian, connections.
`integrators/bssn\_rhs.F90`	The BSSN evolution equations.
`integrators/ko\_diss.F90`	Kreiss–Oliger dissipation.

File	Role
integrators/sommerfeld.F90	Outer boundary condition.
helper/excision.F90	The excision mask, ghost fill, and characteristic check.
helper/excision_sat.F90	Level-set characteristic SAT (available, off).
helper/constraints.F90	Hamiltonian and momentum constraint evaluation.
integrators/time_step.F90	RK4, the algebraic projection, the time loop.
helper/horizon.F90	Horizon location finder.
helper/output.F90	HDF5 output.
drivers/vacuum_kerr.F90	The program: setup, time loop, diagnostics.

1.4.2 Not on the production path

Several substantial modules exist that are *not* used by the production run. Knowing which is which saves a great deal of confusion:

- `helper/fd\_excision.F90`, `helper/fo\_*.F90`, `integrators/fo\_*.F90`, `init/fo\_*.F90` — a first-order-in-time ("FO") reduced formulation with a boundary SAT built on `fo\_sbp.F90`. These are a parallel development track. The SBP module does not currently satisfy the SBP property (see Section 8.1), so this track is not usable.
- `helper/equilibrium\_1pluslog.F90`, `helper/eq\_schwarzschild\_data.F90` — special equilibrium data used only by tests.
- `helper/primitive\_recovery.F90` — hydrodynamics recovery. Not exercised by the vacuum problem, though the BSSN RHS does accept an optional matter source `msrc`.

1.4.3 Tests and diagnostics

`make check` runs the full gate. Separately there are diagnostic programs, each a standalone `program` linked against the same modules:

Target	Purpose
test-cartoon	Verifies the Cartoon reconstruction against known fields.
test-constraint-conv	Hamiltonian-constraint convergence on exact data.
test-momentum-conv	Momentum-constraint convergence; a real gate.
test-horizon-drift	Validates the horizon finder against exact Kerr.
test-gamma-driver-rhs	Checks the gauge RHS is stationary on KS data.
diagnose-ks-stationarity	RHS residual on frozen Kerr–Schild.
diagnose-momentum-point	Stored vs. analytic state, term by term.

Target	Purpose
`diagnose-inner-boundary`	Fill perturbation-depth scan.
`diagnose-ghost-accuracy`	Ghost fill vs. exact; derivative errors.

Chapter 2

The Physics

2.1 From ADM to BSSN

2.1.1 ADM

The starting point is the Arnowitt–Deser–Misner decomposition of spacetime into a spatial metric g_{ab}, an extrinsic curvature K_{ab}, and a lapse α and shift β^a, with

$$\mathrm{d}_\perp \alpha = -\alpha^2 K, \tag{2.1}$$

$\mathrm{d}_\perp = \partial_t + \beta^a \nabla_a$ denoting the derivative along the normal direction to the slices.

2.1.2 The conformal split

BSSN writes the physical metric as

$$g_{ab} = e^{2\tilde{\phi}}\,\tilde{\gamma}_{ab} = e^{4\tilde{\phi}}\hat{\gamma}_{ab}, \tag{2.2}$$

with $\tilde{\phi}$ the conformal factor and $\tilde{\gamma}_{ab}$ the conformal metric. The lapse is split as $\alpha = e^{\tilde{\phi}} A$ (here $\hat{A}$, stored as `IALPHA` with `conformal_factor_mode = 0`, so $\tilde{\phi} = \frac{1}{12}\ln\det g$ and $\hat{A} = \alpha e^{-\tilde{\phi}}$), and the shift into $\hat{A}^a = \hat{g}^{ab}\beta_b$ plus a vector $\hat{\Gamma}^a$.

$\tilde{\phi}$ evolution

$$\partial_t \tilde{\phi} = -\tfrac{1}{6}\alpha K + \beta^a \partial_a \tilde{\phi} + \tfrac{1}{6}\beta^a \partial_a \beta^a, \tag{2.3}$$

which is `rhs_out(IPHI)` in `integrators/bssn\_rhs.F90`.

$\tilde{\gamma}$ evolution

$$\partial_t \tilde{\gamma}_{ab} = -2\alpha \tilde{A}_{ab} + \beta^c \partial_c \tilde{\gamma}_{ab} + \tilde{\gamma}_{ac}\partial_b \beta^c + \tilde{\gamma}_{bc}\partial_a \beta^c - \tfrac{2}{3}\tilde{\gamma}_{ab}\,\partial_c \beta^c, \tag{2.4}$$

with $\tilde{A}_{ab}$ the traceless part of the extrinsic curvature in conformal form, $\tilde{A}_{ab} = e^{-4\tilde{\phi}}\left(K_{ab} - \tfrac{1}{3}K g_{ab}\right)$. Writing $X_{ab} = -\tilde{\nabla}_a\tilde{\nabla}_b\hat{A} + \alpha\tilde{R}_{ab}$ and stripping its trace, $\bar{X}_{ab} = X_{ab} - \tfrac{1}{3}\tilde{\gamma}_{ab}\,\mathrm{tr}\,X$, the conformal extrinsic curvature obeys

$$\partial_t \tilde{A}_{ab} = e^{-4\tilde{\phi}}\bar{X}_{ab} + \alpha\left(K\tilde{A}_{ab} - 2\tilde{A}_{ac}\tilde{A}^c{}_b\right) + \beta^c\partial_c\tilde{A}_{ab} + \tilde{A}_{ac}\partial_b\beta^c + \tilde{A}_{bc}\partial_a\beta^c - \tfrac{2}{3}\tilde{A}_{ab}\,\partial_c\beta^c. \tag{2.5}$$

$S^a_b = K^a_b - \frac{1}{3}K\delta^a_b$ is the trace-reversed extrinsic curvature. The code needs the derivative $\nabla_a\nabla_b\beta^c$, obtained from `S%d2xy`, `S%d2xz`, `S%d2yz` assembled by `fill_d2`.

Algorithm: `compute_rhs_interior`

1. For each evolved point (i,k), set $\det\tilde{\gamma}=1$ and $\tilde{\gamma}^{-1ab}$ from S.
2. Compute the conformal geometry: $\tilde{\gamma}^{-1ab}$, $\tilde{R}^{a}{}_{bc}$, $\tilde{R}_{ab}$ (metric part), $\tilde{R}^{\phi}_{ab}$, $\tilde{\Gamma}^{a}$, $\hat{A}^{a}$, $\tilde{\nabla}_a\tilde{\nabla}_b\hat{A}$.
3. $\partial_t\tilde{\phi}$ from (2.3).
4. $\partial_t\tilde{\gamma}_{ab}$ from (2.4).
5. $\partial_t K=-\tilde{\gamma}^{-1ab}\tilde{\nabla}_a\tilde{\nabla}_b\hat{A}+\alpha\left(\tilde{A}_{ab}\tilde{A}^{ab}+\frac{1}{3}K^2\right)+\beta^a\partial_a K$.
6. $X_{ab}=-\tilde{\nabla}_a\tilde{\nabla}_b\hat{A}+\alpha\tilde{R}_{ab}$, strip the trace: $X_{ab}\to X_{ab}-\frac{1}{3}\tilde{\gamma}_{ab}\,\mathrm{tr}\,X$, then (2.5).
7. $\partial_t\tilde{\Gamma}^{a}$ from $\tilde{\Gamma}^{a}{}_{bc}\tilde{A}^{bc}+\frac{2}{3}\tilde{\gamma}^{ab}\hat{A}^{c}\partial_b\partial_c\hat{A}^{a}$.
8. Gauge: $\partial_t\hat{A}$, $\partial_t\beta^a$ per (2.8) and (2.11).

2.2 Constraints

The code evaluates both, for diagnostics; *neither is projected.*

Hamiltonian, `constraints.F90`

$$\mathcal{H}=\tilde{\gamma}^{ab}\tilde{R}_{ab}+\tfrac{2}{3}K^2-\tilde{A}_{ab}\tilde{A}^{ab},\qquad \tilde{R}_{ab}=\tilde{R}^{\gamma}_{ab}+\tilde{R}^{\phi}_{ab}. \tag{2.6}$$

Momentum, `constraints.F90`, ADM form

$$\mathcal{M}^{a}=\nabla_b S^{ab},\qquad S^{ab}=e^{-4\tilde{\phi}}\left[\tilde{A}^{ab}-\tfrac{2}{3}\tilde{\gamma}^{ab}K\right]. \tag{2.7}$$

Gotcha 1. *The original code assembled* (2.7) *as* $\tilde{\gamma}^{ab}\partial_b\tilde{A}_{ab}-\frac{2}{3}K\partial_b\tilde{\gamma}^{ab}-\ldots$ *using the* conformal *connection. That form is cited in the literature but is not equal to the ADM expression above unless* $\det\tilde{\gamma}=1$ *is imposed. Since the state drifts from* $\det\tilde{\gamma}=1$*, the two differ. Rewriting it in ADM form changed the measured momentum residual from a non-converging* $\approx 1.1\times10^{-1}$ *to a cleanly second-order* $4.1\times10^{-2}\to3.2\times10^{-2}\to2.1\times10^{-2}\to1.7\times10^{-2}$*.*

The correction was made only *in the diagnostic routine. An A/B test that corrupts the evolved state deliberately confirmed the diagnostic was not on the evolution path, so no stability risk was incurred.*

Gotcha 2. *`test_momentum_conv` had no assertion in it. It printed numbers and passed unconditionally. A real gate requiring monotonically decreasing RMS was added, and verified to fail when a sign is flipped.*

2.3 Gauge

$$\textit{1+log:}\quad \partial_t\hat{A}=-2\alpha K+\beta^a\partial_a\hat{A}, \tag{2.8}$$

$$\textit{maximal:}\quad \partial_t\hat{A}=-\alpha K+\beta^a\partial_a\hat{A}, \tag{2.9}$$

$$\textit{frozen:}\quad \partial_t\hat{A}=0. \tag{2.10}$$

$$\textit{legacy:}\quad \partial_t\beta^a=\tfrac{3}{4}B^a, \tag{2.11}$$

$$\partial_t B^a=\partial_t\tilde{\Gamma}^{a}-\eta B^a, \tag{2.12}$$

$$\textit{puncture:}\quad \partial_t\beta^a=\tfrac{3}{4}B^a,\qquad \partial_t B^a=-\eta\left(B^a-\Gamma^a\right),\quad \Gamma^a=s\,\frac{2Mx^a}{r^3}, \tag{2.13}$$

$$\textit{frozen:}\quad \partial_t\beta^a=0. \tag{2.14}$$

Gotcha 3. *In* (2.13)*,* $\partial_t B^a$ *has* no $\partial_t \tilde{\Gamma}^a$ *term: in a* Γ*-driver the driver is driven purely toward the analytic profile. Feeding it the* $\tilde{\Gamma}$ *RHS as well — which this code once did — couples* B^a *to a different variable and lets the shift drift instead of being anchored. The code comments record that fixing this alone made the run* worse *in the legacy mode (inner-boundary failure at step 14055 instead of 18285), which is why the legacy mode is still available but the puncture mode is the default.*

Gotcha 4. *Setting the gauge to frozen (modes 2/2) is mandatory before measuring any discretisation residual. If the lapse or shift is evolving, their right-hand sides contaminate the measurement. Several early conclusions in this project were confounded this way before the control was added.*

Chapter 3

The Discretisation

3.1 The grid

```
real(dp), allocatable :: xg(:)   ! x(1) = 0, x(Nx) = xmax
real(dp), allocatable :: zg(:)   ! z(1) = -zmax, z(Nz) = +zmax
```

Fortran indices run $1..N$*, so* $\mathrm{i} = 1$ *is the symmetry axis* $x = 0$ *and* $\mathrm{k} = 1$ *is* $z = -z_{\max}$*. In the production configuration* $N_x = 48$*,* $N_z = 96$*,* $x_{\max} = z_{\max} = 12\,M$*, giving* $dx \approx 0.2553$*,* $dz \approx 0.2526$*, and* $dy = dx$*.*

Gotcha 5. *The symmetry axis is* $x = 0$ *and the* z*-axis is the spin axis. The code mirrors in* x *via a rotation by* π *in the* xy*-plane, which is the correct axisymmetry-about-*z *operation. It is easy to misread the axis conventions from the variable names;* `valid_x` *treats* $ii < 1$ *as "past the symmetry axis" and synthesises it.*

3.2 The Cartoon method in full

3.2.1 What is stored

Only the half-plane $y = 0$*,* $x \geq 0$*. Given a request for the value at* (X, Y, z_k)*, the code:*

1. *Reduces to the cylindrical radius* $\varpi = \sqrt{X^2 + Y^2}$ *and the azimuth* $\theta = \arctan 2(Y, X)$*.*
2. *Interpolates the stored row* $u(\cdot, k, \cdot)$ *radially onto* ϖ*.*
3. *Rotates scalars (no-op), vectors (one rotation), and tensors (full two-index rotation).*

3.2.2 The radial interpolation

Quadratic, three-point Lagrange:

```
function interp_radial(u, k, varpi) result(fvals)
  j = floor(varpi / dx) + 1
  if (j < 1) j = 1
  if (j > Nx - 2) j = Nx - 2
  x0 = xg(j); x1 = xg(j+1); x2 = xg(j+2)
  L0 = (varpi - x1) * (varpi - x2) / ((x0 - x1) * (x0 - x2))
  L1 = (varpi - x0) * (varpi - x2) / ((x1 - x0) * (x1 - x2))
  L2 = (varpi - x0) * (varpi - x1) / ((x2 - x0) * (x2 - x1))
  fvals = L0 * u(j, k, :) + L1 * u(j+1, k, :) + L2 * u(j+2, k, :)
```

Gotcha 6. *This is* ***second order****, and it is the accuracy ceiling for the entire y-direction. The x- and z-derivatives use fourth-order stencils; the y-derivatives cannot exceed second order. In a curvature calculation where y-second-derivatives enter on equal footing with x- and z-, this mixed order is the reason convergence studies in this code show the y error falling more slowly than the others.*

3.2.3 Rotation

For a vector V with components (V_x, V_y),

$$V'_x = \cos\theta\, V_x - \sin\theta\, V_y, \qquad V'_y = \sin\theta\, V_x + \cos\theta\, V_y. \tag{3.1}$$

For a symmetric tensor,

$$T'_{xx} = c^2 T_{xx} - 2cs\, T_{xy} + s^2 T_{yy}, \tag{3.2}$$
$$T'_{xy} = cs(T_{xx} - T_{yy}) + (c^2 - s^2) T_{xy}, \tag{3.3}$$
$$T'_{yy} = s^2 T_{xx} + 2cs\, T_{xy} + c^2 T_{yy}, \tag{3.4}$$
$$T'_{xz} = cT_{xz} - sT_{yz}, \qquad T'_{yz} = sT_{xz} + cT_{yz}, \qquad T'_{zz} = T_{zz}. \tag{3.5}$$

The rotation is applied to $\tilde{\gamma}$, $\tilde{A}$, $\tilde{\Gamma}$, β, and B. It is not *applied to $\tilde{\phi}$, K, or $\hat{A}$, which are scalars.*

Gotcha 7. `get_neighbor` *takes $y_{\mathrm{val}} = 0$ as a sentinel meaning "plain index shift". This is why x- and z-neighbours — which do not need the y-direction at all — still go through* `get_neighbor`*: it is the single choke point through which every neighbour access passes, and having one choke point is what makes the excision mask easy to enforce consistently.*

3.3 Finite differences and the fallback cascade

3.3.1 Every stencil in the code

Derivative	***Stencil***	***Order***
$\partial_x f$	$[f_{i-2} - 8f_{i-1} + 8f_{i+1} - f_{i+2}]/12dx$	*4th*
	$[f_{i+1} - f_{i-1}]/2dx$	*2nd, centred*
	$[-3f_i + 4f_{i+1} - f_{i+2}]/2dx$	*2nd, forward*
	$[3f_i - 4f_{i-1} + f_{i-2}]/2dx$	*2nd, backward*
$\partial_x^2 f$	$[-f_{i+2} + 16f_{i+1} - 30f_i + 16f_{i-1} - f_{i-2}]/12dx^2$	*4th*
	$[f_{i+1} - 2f_i + f_{i-1}]/dx^2$	*2nd, centred*
	$[f_i - 2f_{i+1} + f_{i+2}]/dx^2$	***1st****, forward*
	$[f_i - 2f_{i-1} + f_{i-2}]/dx^2$	***1st****, backward*
$\partial_{xz}^2 f$	$[f_{++} - f_{+-} - f_{-+} + f_{--}]/4\,dx\,dz$	*2nd*
$\partial_y f$	$[f(\varpi,+dy) - f(\varpi,-dy)]/2dy$	*2nd*
$\partial_y^2 f$	$[f(\varpi,+dy) - 2f_i + f(\varpi,-dy)]/dy^2$	*2nd*
$\partial_{xy}^2 f$	*differencing ∂_y in x*	*2nd*
Kreiss–Oliger	$[f_{-2} - 4f_{-1} + 6f_0 - 4f_{+1} + f_{+2}]$	*2nd*

3.3.2 The fallback logic

For each derivative direction the code asks whether the required neighbours are usable, where usable *means "in bounds and not* `EXCISED`*":*

```
xp_ok  = valid_x(i,k,+1,mask)      ! mask == EXCISED -> unusable
xm_ok  = valid_x(i,k,-1,mask)
xp2_ok = valid_x2(i,k,+1,mask)     ! needs BOTH i+1 and i+2 usable
zm2_ok = valid_z2(i,k,-1,mask)
```

It then takes the highest-order stencil all of whose neighbours are available, cascading down as necessary. The mixed xz, xy, yz derivatives each test their own availability and have their own one-sided fallbacks.

Gotcha 8. *The one-sided* second *derivative $(f_i - 2f_{i+1} + f_{i+2})/dx^2$ is* ***first-order accurate****, not second-order. It is easy to assume symmetry with the one-sided first derivative $\frac{-3f+4f_1-f_2}{2dx}$, which genuinely is second-order. So near a boundary the code degrades to first order in second derivatives while remaining second order in first derivatives — an inconsistency that limits local convergence rate.*

Gotcha 9. `valid_x` *returns* true *for $ii < 1$, on the grounds that* `get_neighbor` *can synthesise the mirror point via the Cartoon rotation. An earlier version returned false, which silently forced a one-sided first-order stencil right on the axis. The current behaviour is correct.*

3.3.3 The variable mask contract

`compute_stencil(u, i, k, mask, S)` *receives a mask with the contract*

```
mask(i,k) == EXCISED   ->  point is inside the excision region, unusable
mask(i,k) == anything else -> usable (physical point or filled ghost)
```

This is why `excision_status` *is re-derived from* `excision_geom` *at the top of every* `extrapolate_excised` *call: carrying last step's classification forward would let a point that was filled last step be treated as reliable when it is not.*

Chapter 4

Boundaries

4.1 Inner: excision

4.1.1 The mask

A point is excised if its ***Kerr radius*** *is below the cutoff:*

```
logical function is_excised(i,k)
  is_excised = (kerr_radius_squared(xg(i), zg(k)) < r_excise**2)
end function
```

with

$$r_{\mathrm{KS}}^2 = \frac{1}{2}\left(\rho^2 - a^2 + \sqrt{(\rho^2 - a^2)^2 + 4a^2 z^2}\right), \qquad \rho^2 = x^2 + z^2. \tag{4.1}$$

In the production configuration $a = 0.8$, $M = 1$, so the outer horizon is at $r_+ = M + \sqrt{M^2 - a^2} = 1.6$ and the inner horizon at $r_- = 0.4$. The default $r_{\mathrm{excise}} = 1.0$ therefore sits between the two horizons — inside the black hole but outside the Cauchy horizon, which is the standard and safe choice.

Gotcha 10. *$r_{\mathrm{KS}} \neq \sqrt{x^2 + z^2}$ whenever $a \neq 0$ and $z \neq 0$. The code base contained ten diagnostic filter sites that used the cylindrical radius where the mask uses the Kerr radius. Because the two disagree, the driver's reported "exterior" worst point could lie* inside *the mask — and at every resolution tested, it did ($r_{\mathrm{KS}} \approx 0.7$–$0.8$). This invalidated a substantial body of earlier conclusions; see Chapter 9.*

4.1.2 The ghost fill

Points inside the mask are never evolved. They are given values so that exterior stencils have something to read. The fill extrapolates the ***perturbation from a frozen background****, not the field itself:*

```
! d1..d4 are (u - background) at 1,2,3,4 cells along a ray
if (excision_order >= 4 .and. all four points usable) then
  u(i,k,:) = b0 + 4.0_dp*d1 - 6.0_dp*d2 + 4.0_dp*d3 - d4   ! order 4
if (excision_order >= 3 .and. first three usable)  then
  u(i,k,:) = b0 + 3.0_dp*d1 - 3.0_dp*d2 + d3             ! order 3
if (excision_order >= 2 .and. first two usable)    then
  u(i,k,:) = b0 + 2.0_dp*d1 - d2                       ! order 2
if (usable_point(status,i+si,k+sk))                 then
  u(i,k,:) = b0 + d1                                   ! order 1
```

The background is set once from the initial data by `set_excision_background`.

Gotcha 11. *Because the background is the exact Kerr–Schild solution and the true Kerr–Schild solution is* stationary*, the perturbation $\delta = u - u_{\mathrm{KS}}$ is identically zero at $t = 0$. Therefore* every *order of extrapolation returns the exact answer, to machine precision. This was measured: the worst ghost-fill error anywhere is* 4.4×10^{-16}.

This has two consequences that took a while to appreciate. First, it explains why the fill order makes almost no difference at early times. Second, it means a static $t = 0$ test of the fill is structurally blind *— any perturbation-depth scan at $t = 0$ that perturbs the background will measure the extrapolation of a non-zero field, which is a different question.*

4.1.3 Onion-peeling and the frozen snapshot

The fill runs in sweeps, and each sweep gates every usability check on a frozen snapshot *of the mask taken before the sweep begins:*

```
status_snapshot = status          ! freeze BEFORE filling anything
do k = 1,Nz
  do i = 1,Nx
    if (status_snapshot(i,k) /= EXCISED) cycle
    call fill_one_point(u,status_snapshot,i,k,filled,order)
    if (.not. filled) cycle
    status(i,k) = GHOST
```

Gotcha 12. *Without the snapshot, a point whose best-aligned outward neighbour is not yet ready falls back,* within the same sweep*, to a less well-aligned direction that chains through points filled moments earlier in that same raster pass. Each link in such a chain reapplies the $[4, -6, 4, -1]$ extrapolation, whose L_1 norm is 15, so a chain a few links deep amplifies an $O(1)$ boundary value into $O(10^4)$ garbage. The snapshot forces genuine onion-peeling: a point only ever uses data resolved as of the end of the previous sweep.*

4.1.4 Positivity enforcement

After filling, any ghost whose metric is not positive definite is blended back toward the background:

```
do n = 1, 40
  theta = 0.5_dp * theta
  trial = (1.0_dp-theta)*gb + theta*g
  if (metric_is_spd(trial)) exit
end do
```

This is a safety net, not a physical model. It did not trigger in any run examined.

4.1.5 The characteristic SAT

`helper/excision\_sat.F90` *implements a level-set boundary SAT. It is compiled and unit-tested, and enabled by* `excision_sat = true` *in the parameter file. It is* off *in production. Tests show it slightly damps the inner boundary (lower* `max|H|`*) but does not change survival, and the puncture gauge is not actually driving anything — see the* `KNOWN LIMITATION` *note at the top of the Makefile.*

4.2 Outer: Sommerfeld

On the outer rows and column the RHS is replaced by a first-order outgoing-wave condition applied to every one of the 24 variables:

$$\partial_t f = -c\,\partial_r f - n\,\frac{f - f_0}{r}, \tag{4.2}$$

with $c = 1$ (in units where the wave speed is unity), f_0 the asymptotic flat-space value (1 for the diagonal metric components and $\hat{A}$, 0 otherwise), and n the asymptotic falloff power.

The legacy condition uses $n = 1$ for every field. Measurement showed that for exact Kerr–Schild data the true powers are far steeper: $\tilde{\gamma} \to 1$ (no $1/r$ term at all), $\tilde{\phi}$ and $\hat{A} \sim r^{-3}$, and K, $\tilde{A}$, $\tilde{\Gamma}$, $B \sim r^{-4}$. Setting `GR_SOMMERFELD_DECAY=1` switches to per-field powers.

Gotcha 13. *The per-variable decay powers were tested and are **worse**: RMS constraint violation degrades by 8.5× when the correct powers are used. The $1/r$ term is not an asymptotic-correctness knob — it acts as a damper, and a weaker damper lets more error in. This was a clean refutation of a plausible hypothesis.*

Gotcha 14. *A single-layer, all-field, first-order Sommerfeld condition is* not *a characteristic boundary condition. It overwrites the RHS of all 24 variables in one layer regardless of which characteristics actually leave the domain. A proper treatment would impose conditions on the incoming characteristic fields only. The apparent second-order convergence in domain size L that this condition exhibits is reflection, not accuracy.*

4.3 Time integration

Classical RK4 with an algebraic projection after each stage:

```
utmp = u + 0.5_dp*dt*k1
call extrapolate_excised(utmp)
call project_bssn_constraints(utmp, stepnum)
... k2, k3, k4 likewise ...
u = u + (dt/6.0_dp)*(k1 + 2.0_dp*k2 + 2.0_dp*k3 + k4)
call project_bssn_constraints(u, stepnum)
```

Each stage applies, in order: the interior RHS, the excision SAT, Kreiss–Oliger dissipation, and the Sommerfeld boundary. Note that there is no *projection after stage 1 — the state entering k_1 was projected at the end of the previous step.*

4.3.1 The timestep

```
dt = cfl * min(dx, dz) / sqrt(2.0_dp)
```

with `cfl = 0.005`, so $dt/dx \approx 0.0035$ — about 1.4% of the classical RK4 advection stability limit. This is extraordinarily conservative and was verified not *to be the problem: the failure is not a time-stepping instability.*

4.3.2 The algebraic projection

```
scale = det**(-1.0_dp/3.0_dp)
u(i,k,IGXX) = scale*gxx;  ... (all six components)
! then remove the trace of Atilde in the same loop
```

Gotcha 15. *The projector was the leading suspect for manufacturing its own constraint violation, since it rescales* $\tilde{\gamma}$ *to* $\det = 1$ *four times per step. It was instrumented (*`GR_PROJ_DIAG`*) and measured. The correction is* $\max|\det(\tilde{\gamma})-1| \approx 10^{-5}$*, growing only linearly, five orders of magnitude below the violation it would have to explain.* ***Falsified.***

Chapter 5

Initial Data

Kerr–Schild, from `kerr_schild_point`*:*

```
rho2 = x*x + y*y + z*z
r2   = 0.5_dp*( (rho2 - a2) + sqrt((rho2 - a2)**2 + 4.0_dp*a2*z*z) )
H    = M * r2 * sqrt(r2) / (r2*r2 + a2*z*z)
l(1) = (r*x + bh_spin*y) / (r2 + a2)
l(2) = (r*y - bh_spin*x) / (r2 + a2)
l(3) = z / r
gij  = I + 2.0_dp * H * outer(l, l)
alpha = 1.0_dp / sqrt(1.0_dp + 2.0_dp*H)
beta_lower = 2.0_dp * H * l
```

The conversion to BSSN variables:

```
twoH = max(detg - 1.0_dp, 0.0_dp)
phi  = log(max(detg,1.0e-300_dp)) / 12.0_dp     ! conformal_factor_mode 0
gtij = exp(-4.0_dp*phi) * gij
Ktr  = sum(gijinv*Kij)
Atij = (Kij - (gij*Ktr)/3.0_dp) / exp(4.0_dp*phi)
```

The extrinsic curvature is obtained by central differences of the analytic metric with an internal step $h = 10^{-4}$, plus a Christoffel-symbol correction to $\partial_a \beta_b$.

5.0.1 The conformal connection

`compute_conformal_connections` *fills* `IGTX..IGTZ` *from*

```
dginv(1,:,:) = (ginvxp - ginvxm) / (2.0_dp*dx)
dginv(2,:,:) = (ginvyp - ginvym) / (2.0_dp*dy)
dginv(3,:,:) = (ginvzp - ginvzm) / (2.0_dp*dz)
Gt(1) = -(dginv(1,1,1) + dginv(2,1,2) + dginv(3,1,3))
```

that is, $\tilde{\Gamma}^a = -\partial_b \tilde{g}^{ab}$.

Gotcha 16. *This is a* ***second-order*** *central-difference path, entirely separate from the fourth-order* `compute_stencil` *that the curvature operator uses. So the state field feeding* $\tilde{R}$*'s* $\tilde{\Gamma} \cdot \partial\tilde{\Gamma}$ *term is two orders less accurate than its neighbours, by construction, and its loop bounds* $i = 2..N_x - 1$*,* $k = 2..N_z - 1$ *carry no excision-mask guard on the read range.*

This was the leading remaining suspect and has now been measured: the error is $9.07 \times 10^{-2} \to 3.05 \times 10^{-2} \to 7.68 \times 10^{-3}$ *for* $N_x = 48, 96, 192$*, i.e. it* ***converges*** *at roughly second order and sits within the range spanned by its neighbours. It is lower-order than its surroundings, but it is not an outlier and does not explain a non-converging error.*

Chapter 6

Constraints and Diagnostics

6.1 The margin trap

This is the single most instructive defect found in the project, and it is worth dwelling on because it invalidated a large body of earlier work.

The driver computes `max|H|(ext)` *over points satisfying*

```
if (sqrt(kerr_radius_squared(xg(i), zg(k))) < r_excise + diag_margin) cycle
```

with `diag_margin` *defaulting to* 1.0*. That correctly excludes* $r_{\mathrm{KS}} < 2.0$*, the region in which the code is known to be inaccurate. But it means the reported number says nothing about* $r_{\mathrm{excise}} < r_{\mathrm{KS}} < r_{\mathrm{excise}} + 1.0$*.*

And that band is exactly where the constraint violation is largest. Sweeping the margin at $N_x = 48$*,* $t \approx 0.36$*:*

`diag_margin`	*monitors from*	`max|H|`	`rms|H|`
2.0	$r_{\mathrm{KS}} = 3.0$	4.06×10^{-3}	3.51×10^{-4}
1.0 (default)	2.0	1.17×10^{-2}	7.10×10^{-4}
0.5	1.5	1.53×10^{-1}	4.09×10^{-3}
0.2	1.2	3.19×10^{-1}	1.09×10^{-2}
0.05	1.05	5.25×10^{-1}	1.60×10^{-2}

A factor of 129 between the far field and one cell outside the mask. The default setting reports the small number and calls it the constraint.

Gotcha 17. *For a long time this was interpreted as "the initial data violate the constraint by* $O(1)$ *at* $t = 0$*, and this slowly propagates outward." The* 1.18 *figure that motivated that reading came from the driver's unfiltered* $x = 0$ *profile, which samples* $r_{\mathrm{KS}} \approx 0.38$ *— deep inside the mask, on filled ghost points. It was an artifact of the radius bug. The qualitative conclusion (a steep near-mask gradient exists) survived; the specific numbers did not.*

6.2 The near-mask band

The driver therefore now reports the band explicitly:

```
! GR_NEAR_MASK_W, default 2.0, 0 disables
if (nm_rr < r_excise .or. nm_rr >= r_excise + near_mask_w) cycle
```

```
nm_npts = nm_npts + 1
nm_Hmax = max(nm_Hmax, abs(Hgrid(i,k)))
nm_Mmax = max(nm_Mmax, sqrt(Mgrid(i,k,1)**2 + Mgrid(i,k,2)**2 + Mgrid(i,k,3)**2))
```

This costs one extra pass over arrays that have already been computed — no additional stencil work, no effect on the evolution. Typical output:

```
 step 700  ...  max|H|(ext)= 1.2656E-02  rms|H|(ext)= 2.0211E-03  alpha(probe)= 0.910348
    near-mask r_KS < 3.0000E+00  n= 192  max|H|= 1.9493E+00  max|M|= 1.4920E+00  worst r=
    1.0227
```

The momentum residual deserves attention: **max|M|** *is* 1.49 *inside the band against* 2.87×10^{-2} *outside — a factor of 52. Since the band is where the fill injects its error into the stencil, a large* $\max|M|$ *there is the signature of a fill/stencil inconsistency rather than a gauge problem. That is a useful discriminator between two failure families that are otherwise hard to tell apart.*

Gotcha 18. *At longer times the near-mask momentum residual does something nothing else in this project does: at* $N_x = 48$ *it develops* isolated spikes reaching 1.2×10^8 *at individual grid points, alternating with moderate values of order* 10^2*, while the exterior residual sits at* 5×10^{-2}*. At* $N_x = 96$ *the spikes do not occur and the near-mask value stays below* 2.9×10^1*. This is the most violent single behaviour measured in the project that is not the conformal-determinant failure itself.*

It is worth being precise about what that does and does not show. The run aborts on a non-positive conformal determinant, not *on the momentum residual, so the spikes are a symptom of a severely ill-conditioned near-mask region rather than a demonstrated cause of the abort. They may be an indicator. But they are a genuine resolution effect — halving* dx *removes them — and they are the strongest evidence yet that something is badly wrong immediately outside the mask.*

Comparing the two resolutions at equal physical time also shows that refinement is not *monotonically improving the exterior constraints: it is* $3.7\times$ *worse in* $\mathrm{rms}|H|$ *at* $t = 0.22$*, better through the middle of the run, and worse again by* $t = 4.44$*. There is therefore no power law relating resolution to accuracy over this range, no resolution budget can be fitted, and "just refine it" is not a supported remedy.*

6.3 The diagnostic programs

6.3.1 diagnose-ghost-accuracy

The most capable instrument in the project. It reports, for any resolution:

1. ***Ghost fill error*** *vs. exact Kerr–Schild, binned by depth inside the mask. Result:* 4.4×10^{-16} *everywhere.*
2. ***max|H| per radial bin*** *outward from the mask, so the spatial structure of the error is visible.*
3. ***Per-derivative errors*** *against high-order central differences of the exact analytic field, for* $\tilde{\gamma}_{xx}$ *and* $\tilde{\phi}$*, in* x*,* y *and* z*.*
4. ***Decomposition*** $H = \tilde{R} + \frac{2}{3}K^2 - \tilde{A}^2$*, split further into* $\tilde{R}^\gamma$ *and* $\tilde{R}^\phi$*. Since the* K *and* $\tilde{A}$ *terms are pointwise algebraic and carry no discretisation error, this localises the problem to* $\tilde{R}$ *immediately.*
5. Γ***-tilde accuracy*** *against the same formula evaluated on the exact analytic metric.*
6. ***A replica of the driver's own*** $t = 0$ ***scan****, so the two can be cross-checked digit for digit.*

6.3.2 diagnose-momentum-point

Prints the stored BSSN state beside the analytic state built independently at the same coordinate, component by component, plus the seven additive pieces of the momentum constraint at a chosen point and at a same-radius control away from the axis. Agreement to 10^{-16} *in the data stage is the precondition for blaming the derivatives instead.*

6.3.3 diagnose-inner-boundary

Perturbs the background and measures how the perturbation propagates into the exterior as a function of fill depth. Shows it bounded and saturating.

6.3.4 A methodological warning

Gotcha 19. *Every number in this project that was measured with a diagnostic operating on the wrong set of points had to be retracted. So did two numbers that were produced by bugs in the diagnostic itself. Specifically:*

- *A second-derivative reference built as* $(f(+h) - f(0))/h^2$ *— a forward difference scaling as* f'/h *— reported a spurious* $|\partial_y^2|$ *error of* $\sim 10^3$ *that looked exactly like the smoking gun. The correct central difference gives* $\sim 10^{-2}$ *and converges normally.*
- *A diagnostic that called* `set_kerr_schild_id` *but not* `compute_conformal_connections` *left* $\tilde{\Gamma}$ *identically zero, so every Hamiltonian it reported was computed with a term of the curvature silently dropped. Adding the call moved the innermost-cell* $\max|H|$ *at* $N_x = 96$ *from* 4.93×10^{-1} *to* 1.14×10^{-1}.

The general lesson, which cost real time: when a measurement contradicts an expectation, suspect the instrument before believing the result, and prefer two independent instruments that agree over one instrument that agrees with your prior.

Chapter 7

Build and Test

7.1 The parity gate

```
check: check-parity $(TARGET) test

check-parity:
        @missing=0; \
        srcs=' $(SOURCES) '; \
        for f in $$(grep -ho '[a-z_]*/[a-z_]*\.F90' Makefile | sort -u); do \
          case "$$f" in drivers/*|helper/test_*|helper/diagnose_*|init/test_*|helper/probe_*)
    continue;; esac; \
          case "$$srcs" in \
            *" $$f "*) ;; \
            *) echo "check-parity: $$f is built by a test but missing from SOURCES"; missing
    =1;; \
          esac; \
        done; \
        ...
```

This is a genuinely good idea and worth copying. It scans the Makefile for every source file any test target compiles, and asserts that each also appears in `SOURCES` *— the list the production binary is built from. Without it, a test can pass against a copy of the physics that production does not actually use, and you would never know.*

7.2 Sources must be listed in dependency order

```
SOURCES := \
        helper/kinds_mod.F90 \
        helper/fo_sbp.F90 \
        ...
        init/param.F90 \
        init/grid.F90 \
        init/vars.F90 \
        ...
        drivers/vacuum_kerr.F90
```

gfortran will not resolve module dependencies automatically across a single invocation, so this list must be topologically ordered. This is a maintenance burden; add new modules before the files that `use` *them.*

Gotcha 20. *The Makefile notes that GNU make predefines* `FC = f77`*, so a plain* `FC ?= gfortran` *never takes effect and every recipe silently builds with* `f77`*. The fix is*

```
ifeq ($(origin FC),default)
FC      := gfortran
endif
```

so that an explicit `make FC=...` *still wins. This is a classic and a good reminder to check what your build system already defines.*

7.3 Environment overrides

Parameters can be overridden at runtime without editing files, which is what makes A/B experiments cheap and reproducible:

Variable	***Effect***
`GR_NSTEPS`	*Number of steps.*
`GR_DIAG_MARGIN`	*Width of the excluded near-mask band.*
`GR_NEAR_MASK_W`	*Width of the reported near-mask band; 0 disables.*
`GR_SLICING_MODE`	*0 = 1+log, 1 = maximal, 2 = frozen.*
`GR_GAMMA_DRIVER_MODE`	*0 = legacy, 1 = puncture, 2 = frozen, 3 = advective.*
`GR_EXCISION_ORDER`	*1–4; the ghost extrapolation order.*
`GR_EXCISION_SAT`	*Enable the level-set characteristic SAT.*
`GR_KO_EPS`	*Kreiss–Oliger strength.*
`GR_GAUGE_ETA`	Γ*-driver damping.*
`GR_REFRESH_BG`	*Re-anchor the excision background every step.*
`GR_SOMMERFELD_DECAY`	*Per-field asymptotic decay powers.*
`GR_PROJ_DIAG`	*Report the algebraic projection correction.*
`GR_HZ_EVERY`	*Horizon-find interval; 0 disables.*
`GR_HZ_ABORT`	*Abort when the horizon drifts.*

Chapter 8

A Tour of Every File

File	*What it does and why*
`helper/kinds_mod.F90`	`dp = selected_real_kind(15,307)`. *One place to change precision.*
`init/param.F90`	*Every runtime parameter with a default, a parser, and an environment override.*
`init/grid.F90`	*Builds* `xg` *and* `zg`*. Fortran 1-based; x starts at 0, z is centred.*
`init/vars.F90`	*The 24 state indices, plus* `ICONFMET/IATILDE` *component tables for looping over symmetric tensors, and* `is_tensor_var/is_vector_var` *for the Cartoon rotation.*
`init/init_data.F90`	*Kerr–Schild point data, conversion to BSSN, and the conformal connection.*
`init/stencils.F90`	*Precomputed finite-difference weights used by the SAT and equilibrium modules.*
`init/fd_weights.F90`	*FD_RADIUS and weight tables.*
`helper/cartoon.F90`	`get_neighbor`*, radial interpolation, vector and tensor rotation.*
`helper/stencil.F90`	*The stencil engine: every derivative, every fallback, the excision-aware validity tests.*
`helper/geometry.F90`	`compute_conformal_geometry`*:* $\tilde{\gamma}$*,* $\tilde{\gamma}^{-1}$*, Christoffels,* $\tilde{R}^{\gamma}_{ab}$*,* $\tilde{R}^{\phi}_{ab}$*,* $\tilde{\nabla}_a\tilde{\nabla}_b\hat{A}$*,* $\hat{A}^a$*.*
`integrators/bssn_rhs.F90`	*The evolution equations. Pure functions of the stencil and geometry; no state mutation.*
`integrators/ko_diss.F90`	*Kreiss–Oliger, applied in all three directions, skipping points inside* $\sqrt{x^2+z^2} < r_{\mathrm{excise}}$*.*
`integrators/sommerfeld.F90`	*Outer boundary condition, with the per-variable decay probe.*
`helper/excision.F90`	*Mask geometry, frozen background, the onion-peeling fill, positivity enforcement, the incoming-characteristic check, and* `kerr_radius_squared` *for external use.*
`helper/excision_sat.F90`	*Level-set characteristic SAT. Compiled and tested; off in production.*

File	*What it does and why*
helper/constraints.F90	*Hamiltonian and momentum, both with optional term decompositions for diagnosis.*
integrators/time_step.F90	*rk4_step and project_bssn_constraints.*
helper/horizon.F90	*Locates the horizon from the state and reports its drift.*
helper/output.F90	*HDF5 snapshots and diagnostics.*
drivers/vacuum_kerr.F90	*Setup, time loop, all reporting, horizon tracking, abort logic.*

8.0.1 The first-order track

File	*Role*
helper/fo_sbp.F90	*SBP operator for the first-order formulation.*
helper/fo_sat.F90	*Simultaneous approximation term.*
helper/fo_constraints.F90	*First-order constraint operators.*
helper/fo_constraint_damping.F90	*Constraint damping.*
helper/fo_curved_sat.F90	*SAT on a curved (non-constant-coefficient) boundary.*
helper/fo_normal_system.F90	*Reduction of the SAT to a normal system.*
helper/fd_excision.F90	*Finite-difference-level excision helpers.*
init/fo_vars.F90, *init/fo_complete_vars.F90*	*Reduced and full state layouts.*
integrators/fo_rhs.F90, *fo_complete_rhs.F90*	*Reduced RHS operators.*

8.1 SBP-SAT: why it is not finished

For a well-posed hyperbolic boundary treatment one wants a simultaneous approximation term built on a discretisation satisfying the summation-by-parts property. The existing `fo_sbp` *module does not: it was machine-checked against the SBP diagonal-dominance test and the maximum eigenvalue of* $hD+D^{\top}h$ *came out at* $+1.685$*, where the property requires it to be non-positive. That is a genuine failure of the property, not a tolerance issue.*

Making this production-ready is a from-scratch build, not a hardening job: a valid SBP operator, the principal symbol of the production system, and the SAT on a curved Kerr-radius boundary, for 24 fields. This is a multi-week project and has deliberately not been started.

Chapter 9

Findings

This chapter records the investigation of the long-time failure, including the mistakes. It is long because the mistakes are as instructive as the results.

9.1 The failure

In the production configuration the run aborts at $t \approx 5.63$ *(step 6331, live gauge) when the conformal metric loses positive definiteness:*

```
Conformal metric failure det= -1.067321E+11 components= ...
Invalid state at step 7536 i=7 k=48 x= 1.53191 z= -0.12632
```

This is not a blow-up in the dynamics; it is the projection's positivity check catching a state that has drifted far from a Lorentzian metric.

9.2 What was established

9.2.1 Negative results, all measured

Hypothesis	***Result***
Frozen gauge	*Falsified. Exact stationary Kerr–Schild gauge still drifts and fails (rate* $4.1\times10^{-2}/t$ *vs* $6.8\times10^{-2}/t$*).*
Outer boundary dominant	*Resolution-dependent. Real and dominant at* $dx \approx 0.10$*; vanishes at the production* $dx \approx 0.25$*. Enlarging the domain* $12M \to 18M$ *at fixed* dx *extends survival 39% but relocates the failure to the inner boundary.*
$1/r$ *Sommerfeld falloff*	*Refuted. Correct per-field powers make the constraint* ***8.5× worse****.*
Cartoon y-derivative	*Measured; scalar and vector exact, tensor* 1.3×10^{-2} *at* $x = dx$*. Only 1–2% of the ring error.*
Algebraic projection	*Falsified.* $\max\lvert\det\tilde{\gamma} - 1\rvert \approx 10^{-5}$*.*
Ghost fill	*Exact to* 4.4×10^{-16}*.*
$\tilde{\Gamma}$ *on the second-order path*	*Converges at* ~ 2*nd order; not an outlier.*
Fill order	*Mitigates, does not fix (see below).*

Hypothesis	***Result***
Stale background	*Null, bit-identical.*
Excision order	*Mitigates only.*
Momentum constraint form	*Was genuinely wrong; fixed, and proven not to be on the evolution path.*

9.2.2 The live picture

The constraint violation is real, large, and confined to the first one or two cells outside the mask. Over physical points only, $\max|H|$ *is* 7.77×10^{-1}, 1.42×10^{-1}, 6.21×10^{-2} *at* $N_x = 48, 96, 192$ *— decreasing, but at observed orders* ~ 2.4 *then* ~ 1.2*, and the worst point sits at a different physical location each time, so the apparent non-convergence may itself be an artifact of comparing maxima at different points.*

Every individual derivative quantity examined converges at roughly second order. The assembled H *exceeds its inputs by a factor of 5–8, consistent with the curvature contraction's* $1/\gamma^2$ *and* $\tilde{\Gamma}\cdot\partial\tilde{\Gamma}$ *terms. The mechanism is therefore* not *localised to a single defective field.*

9.3 Excision order: a trade-off, not a win

Raising `excision_order` *reduces the near-mask violation, but not monotonically in run lifetime:*

`excision_order`	*live fail step*	*live fail t*	*near-mask* $\max|H|$
2	*6331*	*5.63*	***1.289***
3	***6848***	***6.07***	*1.949*
4	*4364*	*3.84*	*2.068*

Order 3 is the production default: it gains 8% in lifetime and improves all three exterior metrics. Order 4 improves the constraint further but costs 31% of survival — a more accurate fill that destabilises. Note the near-mask column runs opposite *to the margin-based column used earlier to justify order 4; the ordering of the three settings depends on which radial band is measured, and that is recorded rather than smoothed over.*

The honest summary is that `excision_order` *is a mitigation worth about a factor of two, and that even the best setting leaves the near-mask error near* 1.1 *against a far-field level of* $\sim 10^{-3}$*. No setting of this knob fixes the problem.*

9.4 How this investigation went wrong, repeatedly

Worth recording in full, because the failure mode is common and expensive.

1. ***Generalising from one regime.*** *The outer-boundary conclusion was drawn at* $dx \approx 0.10$ *and asserted for the code; at the production* $dx \approx 0.25$ *the effect is absent. Every earlier conclusion had been drawn at* $t \approx 0.5$*, while the run dies at* $t \approx 5.6$*.*

2. ***Trusting a diagnostic's default threshold.*** `max|H|(ext)` *with the default margin could not see the region that mattered. Every "null result" measured that way had to be re-examined.*

3. ***Bugs in the instrument producing confident nonsense.*** *Twice, as detailed in Section 6.*

4. ***A wrong radius in a filter.*** *Ten sites. This made a garbage maximum look like a physical result and made the "non-convergence" of* $\max|H|$ *an artifact.*

5. ***Asserting strict dominance from a single metric.*** *Order 3 was recommended as strictly better than order 2 on the strength of one radial band; a second band reversed the near-mask ordering.*

   The through-line is that every conclusion rested on a measurement whose fitness for purpose had not been verified. The single most useful discipline would have been, before interpreting any diagnostic, to confirm it is measuring the quantity you think it is — by cross-checking against a second, independently written instrument.

Chapter 10

Writing Your Own

10.1 Where to start

If you are writing a new code in this style, do it in this order. Each step is independently testable, and each has a natural test that will catch you if you get it wrong.

1. ***Kerr–Schild initial data and the BSSN conversion.*** *Verify the stored state against an independently computed analytic state, component by component. Expect agreement to round-off. If it does not, your conversion is wrong and everything downstream will be too.*

2. ***The stencil engine on a uniform grid with no mask.*** *Write a test that differentiates a manufactured smooth function and confirms the observed order matches the nominal order. Do this for every derivative, including the mixed ones. You will find your mixed-derivative stencils are wrong more often than you expect.*

3. ***Cartoon.*** *Differentiate a manufactured axisymmetric function, including a tensor field, and measure the order. Expect second order in y, and expect it to be worse than x and z because of the radial interpolation.*

4. ***Conformal geometry.*** *Compute $\tilde{R}_{ab}$ two ways: once via the connection-function form used here, and once from Christoffel symbols directly, on a random metric. They must agree to round-off. Getting the index order of* `Chr_low` *wrong is easy and produces a result that is wrong by $\sim 10\%$ without looking obviously wrong.*

5. ***Constraints on exact data.*** *Compute $\mathcal{H}$ and $\mathcal{M}^a$ on exact Kerr–Schild and confirm convergence under refinement. The continuum values are zero, so any non-convergence is entirely your error. This is the single most valuable test in the project and it should come before any evolution work.*

6. ***Excision.*** *Only then add the mask. Confirm that a frozen-gauge configuration stays stationary. If it does not, the problem is somewhere in your fill or stencil fallback, and you can localise it by the bin-by-radius diagnostic described in Section 6.*

7. ***Evolution.*** *Turn the gauge on. Change one thing at a time, and after every change re-run the frozen-gauge stationarity check.*

10.2 Design lessons worth stealing

Keep one copy of the physics. *Every reduced or alternate operator should call the same point-level routine rather than reimplementing it. The codebase does this deliberately:* `compute_rhs_point` *exists precisely so that no second, subtly different BSSN system can grow.*

Gate production against tests. *The* `check-parity` *target is a good pattern: it verifies that what the tests exercise is what production builds.*

Make diagnostics opt-in and inert by default. *Every probe in this codebase is behind an environment variable, and the default path is verified bit-identical with the probe enabled. A diagnostic that changes the answer is worse than no diagnostic.*

Write the regression gate, then verify the gate fails. *Several gates here had to be deliberately broken to confirm they would catch a regression.*

Keep a findings document and date it. `baseline/` *in this repository records each investigation, including the retractions.*

Restate the baseline whenever a diagnostic changes. *Numbers of record are only meaningful relative to the exact code that produced them.*

10.3 Where the physics is genuinely unfinished

In rough order of value:

1. ***The near-mask error.*** *This is the actual bug. It is an $O(1)$ Hamiltonian violation in the first two cells outside the mask, present in the initial discretisation, not caused by the projection, the fill, the connection, or the gauge. It is what kills the run.*
2. ***A real characteristic boundary treatment.*** *The outer Sommerfeld condition is not characteristic and not accurate; the second-order convergence it appears to exhibit is reflection.*
3. ***SBP–SAT.*** *Blocked on a genuinely invalid SBP operator.*
4. ***An active puncture gauge.*** *The* `KNOWN LIMITATION` *note is accurate: the puncture gauge does not currently drive anything, and the horizon drifts.*

Chapter 11

Deriving BSSN, and Why It Exists

This chapter fills in the reasoning that is usually skipped. If you want to write your own code you will eventually want a variable you have not seen before, and the only way to know whether it is necessary is to understand what problem each addition solves.

11.1 The failure of ADM in practice

The ADM system, written on the spatial metric and extrinsic curvature, is a first-order-in-time hyperbolic system whose evolution is constraint preserving in the continuum*. That is, if you start on a surface satisfying the Hamiltonian and momentum constraints, the exact continuum evolution keeps you on that surface.*

The qualification — in the continuum *— is the entire problem. In discrete form the constraints are violated at the truncation error of whatever difference operator you used, the errors grow, they feed back through the equations, and the run eventually fails. This is the constraint-violation / constraint-error feedback mechanism, and it is why naive ADM codes have a reputation for blowing up.*

11.2 What the constraint propagation mechanism needs

For a formulation to damp constraint violations rather than amplify them you need, in rough order of importance:

1. ***A semigroup estimate that propagates the constraint violation away.*** *A violation of size ϵ should lead to a violation of size $\epsilon + O(\Delta t^3)$ — growing at the local truncation rate, not exponentially.*

2. ***Constraint terms of the right size in the evolution equations.*** *The constraint itself should appear in the equations, so that the same mechanism that produces a violation also produces a restoring force.*

3. ***Coupling between adjacent constraint-violation components.*** *A violation at one point should not be able to grow without limit; the diffusion part of the system must dominate at long wavelengths.*

4. ***A parameter family that can be dialled.*** *This is the single most practical of the four, and the reason the theory is parameterised: different schemes are stable over different ranges of the parameter, and the existence of* some *stable regime is what matters.*

ADM fails the second condition badly: the constraints do not appear in the evolution equations at all, so nothing generates a restoring force. Violations grow at the full truncation rate and eventually destroy the metric.

11.3 Where each BSSN piece comes from

Addition	***The problem it solves***
Conformal factor $\tilde{\phi}$ and conformal metric $\tilde{\gamma}_{ab}$	*Divides the metric into a well-scaled part and a scalar. The bulk of the field content goes into $\tilde{\gamma}$ near unity, so the finite-difference stencils see a well-conditioned, slowly varying function rather than a metric that varies strongly. The conformal factor absorbs the rest. This is the single most important structural change, and it is why BSSN tolerates black holes at all.*
Traceless extrinsic curvature $\tilde{A}_{ab}$	*Makes the dynamical degree of freedom trace-free, which is what allows the constraint to appear in the evolution equation for $\tilde{A}$. The trace is carried separately as K, which satisfies a simple, stable equation.*
Connection $\tilde{\Gamma}^a$	*Encodes the conformal connection as an evolved variable rather than a quantity recomputed from first derivatives. Differentiating a connection introduces third derivatives; evolving it introduces none. This is a discretisation-driven design decision, not a mathematical one, and it is the clearest example in the codebase of a variable that exists because of finite differences.*
Driver B^a	*Provides the memory needed to integrate the second-order shift equation $\ddot{\beta}^a = \frac{3}{4}\partial_a B^a$ without effectively making the system third order in the differentiation. The price is that B^a must be evolved too, and the whole family of Γ-driver and B-driver gauges is a choice of how to close that system.*
Shift split $\hat{A}^a$	*The difference between the contravariant and covariant shift is a vector whose magnitude is comparable to the radius. Carrying $\hat{A}^a$ instead of β^a means the evolved quantity is $O(1)$ everywhere rather than growing with radius, which matters for a code that must resolve both the near-hole and far-field regions on one grid.*

Gotcha 21. *Each of these is a* derived *quantity in the continuum, so it is tempting to think of them as convenient reformulations. They are not: they are extra evolution equations with their own truncation errors, their own stability constraints, and their own consistency requirements. Any code that adds a variable must add a test that the variable's analytic value is actually preserved. In this codebase,* `compute_conformal_connections` *is that test for $\tilde{\Gamma}$, and it lives in the production initial-data path because the variable is genuinely evolved.*

11.4 Why the code writes $\tilde{R}$ in connection form

The Ricci tensor of the conformal metric appears in the K and $\tilde{A}$ equations. It can be computed from Christoffel symbols directly, or in the "connection-function" form the code uses, in which the

second derivatives appear only through the Laplacian:

$$\begin{aligned}\tilde{R}^{\gamma}_{ab} = & -\tfrac{1}{2}\tilde{\gamma}^{cd}\nabla_c\nabla_d\tilde{\gamma}_{ab} + \tfrac{1}{2}\left(\tilde{\gamma}_{ac}\partial_b\tilde{\gamma}^{cd} + \tilde{\gamma}_{bc}\partial_a\tilde{\gamma}^{cd}\right) \\ & + \tfrac{1}{2}\left(\tilde{\Gamma}^{c}{}_{ab}\partial_c\tilde{\gamma}_{cd} + \tilde{\Gamma}^{c}{}_{bd}\partial_c\tilde{\gamma}_{ac}\right) - \tilde{\gamma}^{ce}\tilde{\gamma}^{df}\left(\tilde{\Gamma}^{c}{}_{ae}\tilde{\Gamma}^{f}{}_{bd} + \cdots\right). \end{aligned} \tag{11.1}$$

Gotcha 22. *There is an index-order trap here that the code comments specifically warn about, and it is worth spelling out because it is invisible in the output. The lowered-index Christoffel-like object is stored symmetric in its* last two *slots:*

```
Chr_low(k,i,j) = Gam^k_{ij}       ! symmetric in i,j -- NOT in k,i
```

A naive contraction `Chr_low(:,i,j)` *silently computes a different tensor. It agrees with the correct answer only when* $\Gamma^{\mathrm{low}}_{kij}$ *happens to be symmetric under a first-slot swap, which is not true in general. The code comments record that the correct version was verified against a direct from-Christoffels computation on a generic metric to machine precision, while the naive version is wrong at the* $\sim 10\%$ *level. A* 10% *error in an interior quantity is the kind of thing that survives review for months.*

Chapter 12

A Worked Example: the Cartoon Method

The Cartoon method is the piece most newcomers find hardest, so here it is derived from scratch, implemented, and validated — the sequence you should follow when adding it to your own code.

12.1 The problem

You have solved a 2D problem on the half-plane $y = 0$, $x \geq 0$, storing $u(x, y = 0, z)$ for each of 24 fields. Your equations, however, are genuinely three-dimensional: they contain ∂_y and ∂_y^2. You need the field off the plane.

12.2 Step 1: establish the symmetry correctly

For axisymmetry about the z-axis, the isometry is $(x, y, z) \mapsto (x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta,\ z)$. A solution satisfies

$$u(x, y, z) = u(x\cos\theta - y\sin\theta,\ x\sin\theta + y\cos\theta, z). \tag{12.1}$$

For $\theta = \pi$ this maps $(x, y) \mapsto (-x, -y)$, so the stored half-plane $x \geq 0$ determines the whole plane. This is why the code mirrors in x using a rotation by π *and not a reflection: a reflection in x alone is not an axisymmetry of this problem.*

Gotcha 23. *It is easy to write the mirror as a reflection and get a plausible-looking answer, because for* scalar *fields the two agree on the $y = 0$ slice. They diverge as soon as you rotate a vector or tensor. Test the tensor case.*

12.3 Step 2: reduce to radius and interpolate

Write the target point as

$$X = \varpi\cos\theta, \qquad Y = \varpi\sin\theta, \qquad \varpi = \sqrt{X^2 + Y^2}, \quad \theta = \arctan 2(Y, X). \tag{12.2}$$

Then

$$u(X, Y, z) = u(\varpi, 0, z) \tag{12.3}$$

and $u(\varpi, 0, z)$ is, by construction, the function you stored. So the problem reduces to one-dimensional interpolation along x at fixed z.

Quadratic Lagrange on the three points bracketing ϖ:

$$u(\varpi, 0, z) = \sum_{m=0}^{2} L_m(\varpi)\, u(x_m, z), \qquad L_m(\varpi) = \prod_{n \neq m} \frac{\varpi - x_n}{x_m - x_n}. \tag{12.4}$$

Gotcha 24. *The bracketing index needs clamping. The code does* `j = floor(varpi/dx) + 1`*, then clamps to $[1, N_x - 2]$. Without the clamp, a stencil reaching past the outer boundary indexes out of bounds, and depending on how your compiler treats that you get either a crash or a plausible wrong answer. Clamp every bracketing index in every interpolator.*

12.4 Step 3: rotate by rank

Scalars are unchanged. Vectors transform as $V' = RV$. Rank-2 symmetric tensors transform as $T' = RTR^\top$, which expands to the six formulas in Section 6 — or, more safely, you can implement the matrix product and let the compiler expand it.

Gotcha 25. *Implement the rotation as a matrix product for* tensors *rather than writing out the six terms by hand. Hand-expanded index algebra is exactly where the $\sim 10\%$ errors of Section 11 come from. If you do write them out, then test them against the matrix form on random inputs.*

12.5 Step 4: validate, in this order

1. ***Scalar, first derivative.*** *Take $f(x) = x^3$ at $z = 0$. Compare $f(\varpi\cos\theta, \varpi\sin\theta)$ against $f(\varpi) = \varpi^3$. Should agree to round-off — this checks only the interpolation and the rotation of a scalar.*

2. ***Scalar, second derivative.*** *Take $f = e^{-r^2}$ with $r^2 = x^2 + y^2$. Then $\partial_y^2 f = (4y^2 - 2)e^{-r^2}$, computable analytically. Refine dx and confirm* ***second*** *order.*

3. ***Vector.*** *Take a vector field with a known covariant/rotating behaviour and check the rotation in isolation by evaluating at $\varpi = x_i$ so the interpolation is exact and only the rotation acts.*

4. ***Tensor.*** *Same, with a symmetric tensor, against the matrix form.*

5. ***Mixed.*** *Finally check $\partial^2/\partial x \partial y$, which is where the code uses a fallback and where errors hide.*

Chapter 13

Debugging Numerical Relativity Code

This chapter is about method rather than mathematics, and it is included because the investigation recorded in Chapter 9 went wrong more often because of how it was measured than because of what it measured.

13.1 Rule 1: verify the instrument before interpreting it

Before you believe any diagnostic number, confirm that the diagnostic is computing the quantity you think. The cheapest way is to make it computable two different ways and check they agree.

In this project, the decisive technique was building a byte-for-byte replica of the production diagnostic inside a separate program and comparing the numbers. They agreed to every digit — which is what allowed the real bug (a filter using the wrong radius) to be found and localised in one step.

Gotcha 26. *If a standalone instrument and the production code disagree, the disagreement is a bug report about one of them. Do not average, do not pick the one whose answer you prefer, and do not assume the production path is the correct one — in this case it was not.*

13.2 Rule 2: cross-check against an independent truth

Exact Kerr–Schild data gives you an independent truth for the data stage: you can build the BSSN state analytically at any coordinate and compare. Agreement to round-off localises the problem to the derivatives. Disagreement localises it to the conversion. This single test divides a large search space in two.

Likewise, exact vacuum data gives $\mathcal{H} = 0$ and $\mathcal{M}^a = 0$ identically, so any constraint residual is entirely discretisation error and must converge under refinement. A constraint that does not converge is telling you about a defect, not about physics.

13.3 Rule 3: check the order of accuracy you claim

Measure convergence before believing accuracy. The expected order for each stencil is in the table in Section 3.3; if the measured order is lower, you have found something.

Concretely, in this code: the one-sided second derivative is first order while the one-sided first derivative is second, so a boundary-adjacent point sees mixed orders; the Cartoon y-derivatives are second order while x and z are fourth; and $\tilde{\Gamma}$ is computed by a second-order path while its neighbours

use fourth order. All three are real, all three are invisible without an explicit convergence test, and only the third was ever measured.

Gotcha 27. *When you refine, ask whether you are comparing the same physical quantity. In this project the "same" metric — $\max|H|$ over a radial band — selected different physical points at each resolution, because the band is defined in units of dx. An apparent non-convergence can be entirely an artifact of that.*

13.4 Rule 4: know which knobs are inert

Before an A/B test, confirm the knob is connected. A null result is only evidence if the switch demonstrably changes something.

Two examples from this project. A background-refresh switch produced bit-identical *output, which was the correct and useful result — but only because bit-identity was checked, rather than "the numbers look the same". And a projection diagnostic initially reported all zeros, because the field it was differentiating had never been initialised in the diagnostic's own copy of the state; that was an instrument bug, not a physics result.*

Gotcha 28. *Establish the positive control first: make the switch do something obviously different, confirm the diagnostic shows it, then turn it off and confirm the diagnostic goes quiet. Only then is a null meaningful.*

13.5 Rule 5: gates must be able to fail

A test that prints numbers and has no assertion always passes, and will pass forever while the code is broken. This project had one such test (`test_momentum_conv`). The fix is not just to add an assertion but to demonstrate that the assertion fails*, by deliberately breaking the physics it checks.*

The same applies to the parity gate: it exists to catch tests that validate code production does not build. Break the `SOURCES` list deliberately and confirm it complains.

13.6 Rule 6: keep a dated findings log, including retractions

Every investigation in this project produced a markdown file under `baseline/`, dated, and stating what was measured — including the measurements that turned out to be wrong. Several findings had to be retracted.

The retractions are the valuable part. A project that deletes its mistakes leaves behind a set of confident claims with no indication of which were shaky, and the next person repeats the work. Writing "this was wrong, and here is why the measurement was invalid" is worth more than the original conclusion was.

13.7 Rule 7: restate the baseline whenever anything moves

Numbers of record are meaningful only relative to the exact code that produced them. When a diagnostic changes what it measures, or a parameter default changes, the baseline must be re-measured and the old value annotated — not quietly replaced.

In this project the baseline was restated twice: once when the diagnostic radius was corrected, and once when `excision_order` changed. Both times the previous value is recorded in the commit message and the findings document.

Chapter 14

Where the Numbers Come From

Every quantitative claim in this document, and where it was measured.

14.1 The baseline of record

Production configuration, $N_x = 48$, $N_z = 96$, live gauge, `excision_order = 3`*, 700 steps:*

`max|H| (ext)`	1.2656×10^{-2}
`rms|H| (ext)`	2.0211×10^{-3}
`max|M| (ext)`	2.8728×10^{-2}
`alpha(probe)`	0.910348
near-mask `max|H|`	$1.9493 \times 10^{+0}$
near-mask `max|M|`	$1.4920 \times 10^{+0}$

14.2 Lifetimes

Configuration	*fail step*	*fail t*	*fail* r_{KS}
12M, N_x=48, frozen	*7536*	*6.73*	*1.315*
18M, N_x=72, frozen	*10495*	*9.34*	*1.015*
order 2, live	*6331*	*5.63*	—
order 3, live	*6848*	*6.07*	—
order 4, live	*4364*	*3.84*	—
order 2, frozen	*7536*	*6.70*	—

The frozen order-2 entry reproduces a previously recorded value exactly, which is what validates the measurement harness.

14.3 Near-mask measurements

Quantity	*value*	*note*
Ghost fill error (worst anywhere)	4.44×10^{-16}	*machine precision*
Projection $\max\lvert\det\tilde{\gamma} - 1\rvert$	$9.8 \times 10^{-6} \to 2.3 \times 10^{-5}$	*400 steps*
$\tilde{\Gamma}$ *error at* $dr/dx < 1$	$9.07 \times 10^{-2} \to 7.68 \times 10^{-3}$	*N_x 48, 96, 192*

14.4 Driver worst-point locations, corrected filter

N_x	*driver* `max|H|`	*replica*	*worst* r_{KS}	*status*
48	8.2028×10^{-1}	*same*	*0.8255*	GHOST
96	$1.8057 \times 10^{+0}$	*same*	*0.8125*	GHOST
192	5.0928×10^{-1}	*same*	*0.7092*	GHOST

Every reported worst point was inside the excision region. After the fix, the same scan reports 1.86×10^{-2} *at* $r_{\mathrm{KS}} = 2.158$.

Appendix A

Traps, Collected

Every one of these cost real time in this project.

#	**Trap**
1	*Kerr radius ≠ cylindrical radius. Use* `kerr_radius_squared` *for anything that must agree with the mask.*
2	*A diagnostic that excludes a band is blind in that band. State what it covers, and report the excluded region separately.*
3	*One-sided second derivatives* $(f_i - 2f_{i+1} + f_{i+2})/h^2$ *are* first-*order accurate.*
4	*Extrapolate the perturbation from a background, not the field. Extrapolating the field amplifies its own truncation error.*
5	*Freeze the mask snapshot at the start of each fill sweep, or chains within a sweep amplify by* 15^n.
6	`get_neighbor` *treats* $y_{\mathrm{val}} = 0$ *as a sentinel for a plain index shift.*
7	`Chr_low` *is symmetric in its* last *two slots. Contracting against the first silently computes the wrong tensor — wrong by* $\sim 10\%$ *and not otherwise suspicious.*
8	*Never freeze only one of lapse/shift while measuring discretisation residuals.*
9	*A forward difference is not a second derivative. It scales as* f'/h *and will produce spectacular, entirely fictional results.*
10	*A test with no assertion always passes.*
11	`FC ?= gfortran` *never fires in make;* `FC` *is predefined.*
12	*Fortran is case-insensitive:* `Q/q` *and* `Gam/gam` *collisions cause repeated compile failures.*
13	*Fortran I/O is line-buffered when connected to a terminal but block-buffered when redirected. Use* `stdbuf -oL` *for long runs.*
14	*A test that honours its argument versus one that is hard-wired to a resolution will silently waste a run.*
15	`make check` *failing for a pre-existing reason trains you to ignore it.*
16	*Distances in cells versus in* M *are not interchangeable near a mask; a bin labelled "*$\mathrm{d}r/\mathrm{d}x$*" is a different set of points at every resolution.*

Appendix B

Environment Overrides and Reproduction

Command	*What it reproduces*
`make check`	*The gate: parity, production build, full test suite.*
`make run`	*The production configuration.*
`./bin/diagnose\_ghost\ _accuracyparameter\ _files/vacuum\_kerr.par`	*Ghost-fill accuracy, derivative errors, H decomposition, $\tilde{\Gamma}$ accuracy.*
`GR\_DIAG\_MARGIN=0.05. /bin/vacuum\_kerr\ ldots`	*The margin sweep.*
`GR\_NEAR\_MASK\_W=0. /bin/vacuum\_kerr\ ldots`	*Silences the near-mask line, to confirm it is inert.*
`GR_SLICING_MODE=2 GR_GAMMA_DRIVER_MODE=2 ...`	*Frozen Kerr–Schild gauge; the stationary control.*
`GR\_PROJ\_DIAG=1. /bin/ vacuum\_kerr\ ldots`	*Projection correction report.*
`GR_EXCISION_ORDER=n ...`	*The order sweep.*
`GR_SOMMERFELD_DECAY=1 ...`	*Per-field asymptotic powers.*

Environment. *gfortran with* `-fcheck=all -fbacktrace` *for the test builds;* `h5fc` *for HDF5; a TeX distribution for this document. The verification commands used throughout are* `make check` *for the suite and the 700-step run for the baseline of record:*

```
GR_HORIZON_EVERY=0 GR_HZ_ABORT=0 GR_NSTEPS=700 \
  ./bin/vacuum_kerr parameter_files/vacuum_kerr.par
```